

COMP 6231: Distributed Systems

Assignment 1

Architectural Design and Trade-Off Analysis of a Real-Time Ride-Sharing Platform Backend

Author: Travis Holt
Student ID: 40351596
Instructor: Dr. Reza Mirsalari
Date: July 2, 2026

1 Introduction

1.1 Problem Statement

"...architect the backend infrastructure of a real-time ride-sharing platform similar to Uber or Lyft, using principles from distributed systems. Your task is to analyze and design a distributed backend system that supports high availability, low latency, and data consistency under high concurrency."

You must integrate and analyze the following core architectural components

- Database Sharding (Hash-based, Range-based, or Consistent Hashing)
- Replication and Consistency Models (Eventual, Strong, Quorum-based)
- Load Balancing (Layer 4 vs. Layer 7)

1.2 System Requirements

1.2.1 Functional Requirements

- Drivers broadcast live location and availability.
- Riders send trip requests that are matched with nearby drivers.
- The system supports live matching, route calculation, and payment processing.
- Data includes user accounts, trip histories, real-time GPS data, and pricing rules.

1.2.2 Non-Functional Requirements

- Global scale support (multi-region architecture assumed).
- Sub-200ms latency for ride-matching response in 90% of requests.
- High read/write throughput for hotspots (e.g., downtown during peak hours).

1.3 Report Organization

The proposed system is introduced graphically with a series of architecture diagrams with increasing granularity. Design considerations, trade-offs, and decisions are discussed subsequently. The report concludes with recommendations for implementing this system.

2 System Architecture Overview

2.1 System Diagrams

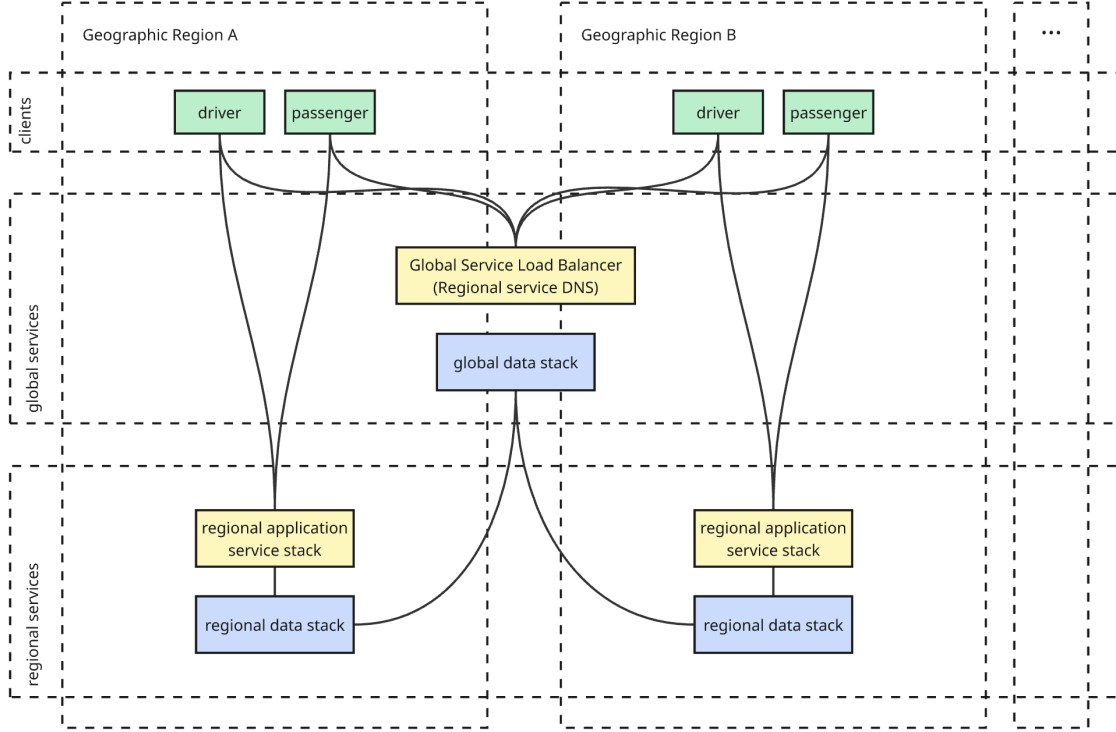


Figure 1: High-level system architecture.

At the highest level the system components can be categorized into three groups: clients, global services, and regional services. The personas of the clients may be either passengers or drivers and their interfaces may be native mobile applications or web browsers. The global services are responsible for authentication, authorization, account management, and regional service routing. The bulk of core functionality is provided by the regional services, of which there can be an arbitrary number of instances.

When clients begin interacting with the system they query the global service load balancer (GSLB) to determine the appropriate regional service endpoint with which to communicate. The GSLB's DNS resolution is governed by a geolocation routing policy that maps a client's current IP address to a regional service gateway's IP address. This policy may also consider availability, capacity, and load.

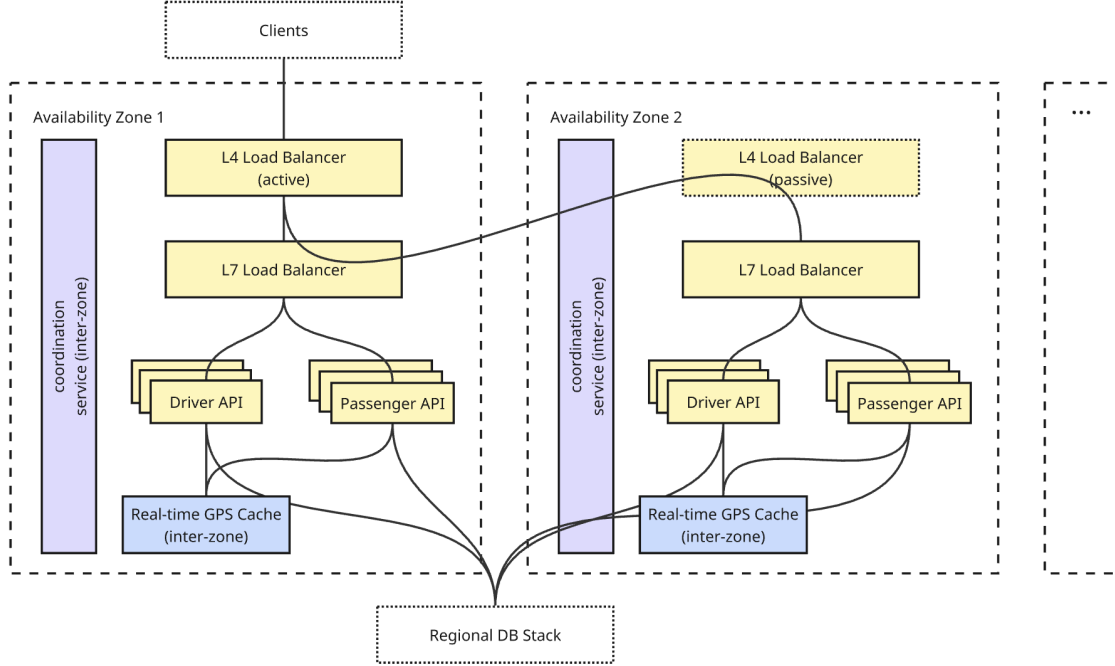


Figure 2: Regional application service stack architecture.

Once a client has been routed to a regional service endpoint a layer 4 load balancer (L4LB) proxies their requests to an availability zone (AZ). The purpose of the L4LB is to distribute load across AZs and adapt to outages in AZs. At any given time there is a single active L4LB in a region, backed by instances that can take over should the active instance fail. Next, a layer 7 load balancer (L7LB) proxies requests to an appropriate application API based on the request path and headers. The L7LB is also responsible for TLS termination and rate limiting. This allows components deeper in the stack to focus on core functionality rather than security and load management.

Each application API can have multiple instances running, with the specific number scaling with the current or anticipated load. The API services are stateless which allows them to be regarded as ephemeral. The diagram depicts two specific APIs for illustration purposes, but in practice there will likely be many more along with additional supporting microservices. Each region also has an inter-zone distributed cache for rapidly changing non-durable data such as GPS locations of users. Durable data is managed by regional and global data stacks. Finally, each region has a coordination service (CS) that spans AZs. The CS is responsible for leader election in the case of the L4LBs. It also provides service registration and discovery, enabling the L7LB to route to a flexible pool of API instances.

Each regional data stack manages a distributed wide-column store cluster of high volume active ride data. Each cluster spans the AZs of its region, with peer nodes distributed across them. We use the H3 geospatial cell index as the partition key. The wide-column store distributes rows across nodes via consistent hashing with virtual nodes, which balances load evenly and minimizes data movement when the cluster is resized. Unlike a leader-follower database, the cluster is masterless: any node can accept reads and writes. Data is replicated to a configurable number of nodes per AZ for durability. Consistency is enforced

at query time using a local quorum policy, requiring a majority of replicas within the region to acknowledge an operation before it is considered successful.

Applications are insulated from the complexity of the regional and global data stacks by a DB proxy (DBP). The DBP embeds a database driver that handles token-aware routing to the correct cluster node based on the partition key. The DBP’s primary responsibilities are managing the connection pool and lazily loading records from the global data stack when new clients use the service or existing clients move between regions.

Each regional data stack has a coordination service used by the DBP for service discovery. Since the wide-column store cluster is masterless and uses gossip for internal node coordination, the CS is not involved in data node management. The DBP uses the CS to track which cluster nodes are healthy and should receive connections.

The global data stack is responsible for owning and managing durable data that is not specific to one single region. This data is much smaller in size and slower changing than the regional ride-based data, but arguably more critical to the system’s operator: account information, payment information, and pricing rules for example. The global data stack is based around a single NewSQL cluster. This cluster has a number of replicated data nodes distributed across regions and AZs. This approach allows for quick reads and slow writes, which is fitting for the access pattern expected for this data.

2.2 Interaction Flows

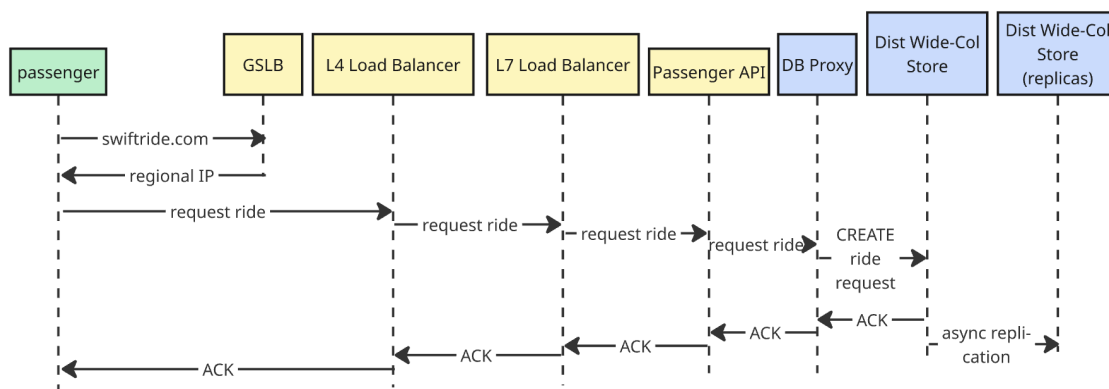


Figure 3: Ride request processing flow.

The diagram traces a simplified ride request from a passenger. The full ride matching flow is more complex and is not shown in its entirety. A rider submits a trip request from their client. The request enters the regional stack through the L4LB and is routed by the L7LB to the ride matching service. The matching service executes a geospatial radius query against the GPS cache to identify available local drivers. From the candidates, the closest available driver is selected and tentatively assigned. The matching service then writes an active trip record to the wide-column store, recording the passenger, the driver, and the initial trip state. Once the write is acknowledged by a quorum, the driver is notified and the trip is live.

3 Database Sharding

3.1 Sharding Strategies

3.1.1 Considered

- **Hash-based:** Assigns each record to a shard by hashing a key (e.g. user ID) and taking the result modulo the shard count. Writes spread evenly and routing is cheap to compute. However, adding or removing a shard changes the modulus, forcing a reshuffle of most existing data.
- **Range-based:** Assigns contiguous key ranges (e.g. timestamps) to specific shards. Range scans are cheap and the shard boundaries are easy to reason about, but it is susceptible to hotspotting. For example a single popular range, like downtown at rush hour, can lead to a single shard absorbing a disproportionate amount of the load. When a shard is removed, only its range needs to migrate to a neighboring shard. The rest of the dataset is untouched. The absorbing shard now owns a larger range, however, which can increase hotspot risk and may require a manual range split to rebalance.
- **Consistent hashing**[4]: Places shards and keys on the same hash ring, so adding or removing a shard only remaps the keys next to it on the ring rather than the whole dataset. It can still hotspot, since one shard may still own a disproportionate arc of the ring, but virtual nodes mitigate this by spreading each physical shard’s ownership across many smaller arcs.

3.1.2 Chosen

The data in this system is partitioned at multiple levels. First, active ride data is isolated by region. We assume that regions have enough flexibility and scaling capacity that their count will rarely change. This is a form of range-based sharding where client IP addresses map to fixed regions. Second, within regional data stacks we shard data by geospatial index (H3[7]) using consistent hashing. This allows the number of shards to scale in response to demand without reshuffling the entire dataset, while also giving us shard keys that are semantically meaningful. As long as the granularity of the H3 cells is chosen appropriately, this approach will also avoid hotspotting. Range queries, such as finding all available drivers within a 2km radius, translate naturally to H3. Each cell has a defined area at a given resolution, so a radius query becomes a lookup over a small fixed set of neighboring cell IDs rather than a full scan.

4 Replication and Consistency Models

4.1 Consistency Model Comparison

4.1.1 Considered

- **Strong consistency:** All reads reflect the most recent write. Strong correctness guarantees and high latency

- **Eventual consistency:** Writes are acknowledged quickly and replicas converge over time. Reads may return stale data
- **Quorum-based consistency:** A write (or read) is acknowledged once a majority of replicas confirm it. A tunable hybrid of the preceding approaches.

4.1.2 Chosen

Each of these consistency models are represented in this system. We use quorum-based consistency for active trip data. A majority of replicas within the region must acknowledge a read or write before it is considered successful, giving us strong correctness guarantees without requiring cross-region coordination. We can accept the modest write latency cost because all replicas involved are within the same region. The distributed GPS cache is eventually consistent, which is acceptable for data that is written frequently, read frequently, and does not involve accounting for real world resources: vehicles dispatched, rides cancelled, payment receipts, etc. Stale reads will not have a meaningfully negative impact on user experience. In return for the reduced consistency guarantee we get lower latency and very high throughput. This is ideal for the data that will be written orders of magnitude more often than other data types in the system. Finally, we distinguish between two categories of global data. Security-critical user account fields (credentials, account status, payment methods, and driver approval status) require strong consistency globally. A deactivated driver who can still accept rides because a regional cache has not caught up is a safety issue, not a UX problem. These fields must always be read through to the global data stack, bypassing the regional DBP cache entirely. Profile data (display name, preferences, photo) can tolerate quorum-based consistency; stale reads have no meaningful consequence, and the latency cost of strong global consistency on every profile read is not justified.

5 Load Balancing

The system contains load balancers performing 3 distinct functions. The GSLB acts as a DNS resolver. Its only purpose is to direct clients to the correct region. It performs this task by mapping client IP addresses to fixed virtual IP (VIP) addresses used by L4LBs in each region. When a L4LB wants to move from passive to active it assumes the VIP.

L4LBs are the gateways for all inbound client traffic to each region. They act as true load balancers, distributing load across available downstream resources as efficiently as possible. They do not process requests or even investigate payloads. They operate at the level of the TCP protocol so the CPU usage is kept to a minimum.

Like the L4LBs, the L7LBs proxy requests towards application services, but L7LBs perform more middleware actions on the requests. Working at the HTTP/application level, these load balancers have access to request paths, methods, and headers. They use this information to filter, modify, and route requests to different types of APIs.

6 Evaluation

6.1 Capacity and Latency

Table 1: Component throughput and latency BOTE estimates based on a survey of candidate implementations. Throughput values are given for a single instance of each component. Latency values are medians.

Component	Technology	Throughput	Latency	Scaling
GSLB	DNS load balancer	~500K queries/s	<1ms	Horizontal with virtual IP
L4LB	L4 load balancer	~200K connections/s	<1ms	Vertical. Horizontal with AZs
L7LB	L7 load balancer	~100K req/s	5ms	Vertical. Horizontal with AZs
CS	Distributed coordination service	~50K ops/s (<i>cluster</i>)	10ms	Fixed (not directly impacted by load)
DBP	Custom	~1M req/s	<1ms	Horizontal
GPS Data Cache	Distributed key-value store	~1M ops/s	<1ms	Horizontal
Active Trip Data	Distributed wide-column store	~100K writes/s	5ms	Horizontal
User Accounts	NewSQL database	~50K TPS	100ms	Horizontal
Trip History	Relational database	~50K reads/s	10ms	Vertical for primary/write. Horizontal for reads

6.2 CAP Theorem

The CAP theorem[2] states that a distributed system can guarantee at most two of three properties:

- **Consistency:** all nodes see the same data at the same time
- **Availability:** every request receives a response
- **Partition Tolerance:** the system continues operating despite network partitions

Since network partitions are a physical reality in any distributed system, partition tolerance is non-negotiable. The remaining choice is between consistency and availability when a partition occurs.

This system chooses CP vs. AP differently depending on data types. The GPS cache is AP. During a partition, nodes continue accepting reads and writes using whatever data they have. A driver’s location being a few seconds stale does not affect safety or correctness. The matching process self-corrects on the next broadcast. Availability matters more here than precision.

The active trip data store is CP. We require local quorum acknowledgment before a write is considered successful. If a partition isolates enough replicas to break quorum, writes will fail rather than proceed with an inconsistent state. A conflict in trip state (e.g. two nodes believing different drivers are assigned to the same rider) is not recoverable and is far worse than a temporary write failure.

The global user account store is also CP. Security-critical fields such as account status, credentials, and payment methods must be consistent everywhere before a change is readable. A deactivated driver account that remains active in a partitioned region is a safety issue. We accept that writes to these fields may be rejected during a global partition rather than risk allowing stale reads.

7 Technology Recommendations

Thus far the system has been described in terms hardware and software agnostic to the extent possible. The system as-is can be implemented with various technologies. In the process of selecting specific environments and frameworks, some singular components may split into multiple, while others may be combined. What follows is an exercise that illustrates how this system might be implemented.

We have chosen to use open-source technologies where possible, to avoid vendor lock-in and to allow for greater flexibility in deployment. The following table summarizes the software recommendations for each system component, along with rationale for each.

7.1 Infrastructure

We will use a cloud provider for the underlying physical infrastructure. This allows us to get up and running quickly with the ability to scale along the way. The specific provider is not important as no provider specific services will be used. (In reality, this may not be the best decision. This was done to allow for learning about open source technologies.)

The compute and networking resources needed to run the system will be provisioned with Terraform. After provisioning, the system will be deployed and configured with Ansible. These two tools allow us to define the system architecture with code instead of manually standing up the infrastructure: it’s fast, readable, and repeatable.

The app services (e.g. APIs) will require more active management than Ansible is intended to provide. There we will containerize the services and manage the containers with Kubernetes [9]. Kubernetes can be configured to automatically scale with load and we also get metrics and logging out of the box.

7.2 Software

Table 2: Software recommendations by system component.

Component	Recommendation	Rationale
GSLB	PowerDNS [11]	Supports geolocation-based DNS resolution and health-check-driven failover
L4LB	HAProxy [8]	High-throughput TCP proxy with active/passive failover. Adds negligible latency.
L7LB	Envoy [5]	Path and header-based routing, TLS termination, rate limiting, and built-in observability.
CS	etcd [6]	Raft-based consensus purpose-built for leader election and service discovery. Proven at scale.
DBP	Custom service	H3 token-aware routing, shard discovery via the CS, and lazy cross-region loading are all app-specific. No DB proxy handles this combination out-of-the-box.
GPS Data	Redis [12]	Supports geospatial commands (<code>GEOADD</code> , <code>GEORADIUS</code>) and per-key TTL. Sub-millisecond latency at high write volume.
Pricing Rules	Redis	Fits entirely in memory. Replication provides regional availability; TTL invalidation handles infrequent updates.
Active Trip Data	Apache Cassandra [1]	Masterless, horizontally scalable across AZs. Tunable consistency supports the local quorum policy chosen for trip records.
User Accounts	CockroachDB [3]	Serializable isolation satisfies the strong consistency requirement. Geo-partitioning anchors records to their home region for fast local reads.
Trip History	PostgreSQL [10]	Time-based range partitioning aligns directly with the sharding strategy for this data. Mature SQL covers the analytical and auditing queries expected here.

References

- [1] Apache Cassandra – the open source distributed database. <https://cassandra.apache.org/>. Accessed: 2026-06-30.
- [2] CAP Theorem – brewer’s theorem. https://en.wikipedia.org/wiki/CAP_theorem. Accessed: 2026-06-30.
- [3] CockroachDB – the distributed SQL database. <https://www.cockroachlabs.com/>. Accessed: 2026-06-30.

- [4] Consistent Hashing – a distributed hash table algorithm. https://en.wikipedia.org/wiki/Consistent_hashing. Accessed: 2026-06-30.
- [5] Envoy – cloud-native high-performance edge and service proxy. <https://www.envoyproxy.io/>. Accessed: 2026-06-30.
- [6] etcd – a distributed, reliable key-value store for the most critical data of a distributed system. <https://etcd.io/>. Accessed: 2026-06-30.
- [7] H3 – hexagonal hierarchical spatial index. <https://h3geo.org/>. Accessed: 2026-06-30.
- [8] HAProxy – the reliable, high performance TCP/HTTP load balancer. <https://www.haproxy.org/>. Accessed: 2026-06-30.
- [9] Kubernetes – production-grade container orchestration. <https://kubernetes.io/>. Accessed: 2026-06-30.
- [10] PostgreSQL – the world’s most advanced open source relational database. <https://www.postgresql.org/>. Accessed: 2026-06-30.
- [11] PowerDNS – the authoritative DNS server. <https://www.powerdns.com/>. Accessed: 2026-06-30.
- [12] Redis – the most popular open source in-memory data store. <https://redis.io/>. Accessed: 2026-06-30.